

EX. NO: 1 IMPLEMENTATION OF DATA ENCRYPTION STANDARD (DES)

AIM

To implement the Data Encryption Standard (DES) encryption and decryption algorithm using Java.

DESCRIPTION

Data Encryption Standard (DES) is a symmetric key block cipher that encrypts data in 64-bit blocks using a 56-bit secret key. It performs a sequence of permutations, substitutions, and XOR operations through 16 rounds of processing. The same key is used for both encryption and decryption.

ALGORITHM

STEP-1: Read the plain text message from the user.

STEP-2: Generate a DES secret key.

STEP-3: Initialize the DES cipher in encryption mode.

STEP-4: Encrypt the plain text.

STEP-5: Display the encrypted text.

STEP-6: Initialize the cipher in decryption mode.

STEP-7: Decrypt the encrypted text.

STEP-8: Display the original message.

CODE

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.util.Base64;
import java.util.Scanner;

public class DESExample {

    public static void main(String[] args) {
        try {

            Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter the message: ");

String message = sc.nextLine();


// Generate DES Key

KeyGenerator keyGenerator = KeyGenerator.getInstance("DES");
SecretKey secretKey = keyGenerator.generateKey();


// Encryption

Cipher cipher = Cipher.getInstance("DES");
cipher.init(Cipher.ENCRYPT_MODE, secretKey);

byte[] encryptedBytes = cipher.doFinal(message.getBytes());

String encryptedText = Base64.getEncoder().encodeToString(encryptedBytes);

System.out.println("\nEncrypted Message:");
System.out.println(encryptedText);


// Decryption

cipher.init(Cipher.DECRYPT_MODE, secretKey);

byte[] decryptedBytes = cipher.doFinal(encryptedBytes);

String decryptedText = new String(decryptedBytes);

System.out.println("\nDecrypted Message:");
```

```
        System.out.println(decryptedText);

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}
```

Output

Clear

Enter the message: run

Encrypted Message:
HG8N2LO3uGc=

Decrypted Message:
run

=== Code Execution Successful ===

RESULT

Thus, the Data Encryption Standard (DES) encryption and decryption algorithm was implemented successfully using Java.

EX. NO: 2 IMPLEMENTATION OF DOUBLE DATA ENCRYPTION STANDARD (2DES)

AIM

To implement the Double Data Encryption Standard (2DES) encryption and decryption algorithm using Java.

DESCRIPTION

Double Data Encryption Standard (2DES) is an enhancement of the DES algorithm. It applies the DES encryption algorithm twice using two different secret keys. The plaintext is first encrypted using the first key (K1), and the resulting ciphertext is encrypted again using the second key (K2). During decryption, the reverse process is performed using the keys in reverse order. Compared to single DES, 2DES provides improved security against brute-force attacks.

ALGORITHM

- STEP-1:** Read the plaintext message from the user.
- STEP-2:** Generate two DES secret keys (K1 and K2).
- STEP-3:** Encrypt the plaintext using Key K1.
- STEP-4:** Encrypt the resulting ciphertext using Key K2.
- STEP-5:** Display the final encrypted message.
- STEP-6:** Decrypt the ciphertext using Key K2.
- STEP-7:** Decrypt the resulting data using Key K1.
- STEP-8:** Display the original plaintext.

CODE

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.util.Base64;
import java.util.Scanner;

public class DoubleDES {

    public static void main(String[] args) {
```

```
try {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter the message: ");

    String message = sc.nextLine();

    // Generate two DES keys
    KeyGenerator keyGen = KeyGenerator.getInstance("DES");
    SecretKey key1 = keyGen.generateKey();
    SecretKey key2 = keyGen.generateKey();

    // First Encryption
    Cipher cipher1 = Cipher.getInstance("DES");
    cipher1.init(Cipher.ENCRYPT_MODE, key1);
    byte[] firstEncrypt = cipher1.doFinal(message.getBytes());

    // Second Encryption
    Cipher cipher2 = Cipher.getInstance("DES");
    cipher2.init(Cipher.ENCRYPT_MODE, key2);
    byte[] secondEncrypt = cipher2.doFinal(firstEncrypt);

    String encrypted = Base64.getEncoder().encodeToString(secondEncrypt);

    System.out.println("\nEncrypted Message:");
    System.out.println(encrypted);
```

```

// First Decryption
cipher2.init(Cipher.DECRYPT_MODE, key2);
byte[] firstDecrypt = cipher2.doFinal(secondEncrypt);

// Second Decryption
cipher1.init(Cipher.DECRYPT_MODE, key1);
byte[] original = cipher1.doFinal(firstDecrypt);

System.out.println("\nDecrypted Message:");
System.out.println(new String(original));

} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

Output

Clear

```

Enter the message: running

Encrypted Message:
NoJSH43EkMPPpw1DPsXpUg==

Decrypted Message:
running

=== Code Execution Successful ===

```

RESULT

Thus, the Double Data Encryption Standard (2DES) encryption and decryption algorithm was implemented successfully using Java

EX. NO: 3 IMPLEMENTATION OF TRIPLE DATA ENCRYPTION STANDARD (3DES)

AIM

To implement the Triple Data Encryption Standard (3DES) encryption and decryption algorithm using Java.

DESCRIPTION

Triple Data Encryption Standard (3DES) is an enhanced version of the DES algorithm that applies the DES encryption process three times using either two or three different secret keys. It provides much stronger security than DES by increasing the effective key length. The most common operation is Encrypt–Decrypt–Encrypt (EDE). Triple DES is widely used in banking systems, secure communications, and financial transactions.

ALGORITHM

- STEP-1:** Read the plaintext message from the user.
- STEP-2:** Generate a Triple DES (DESede) secret key.
- STEP-3:** Initialize the Triple DES cipher in encryption mode.
- STEP-4:** Encrypt the plaintext using the secret key.
- STEP-5:** Display the encrypted ciphertext.
- STEP-6:** Initialize the cipher in decryption mode.
- STEP-7:** Decrypt the ciphertext using the same secret key.
- STEP-8:** Display the original plaintext.

CODE

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.util.Base64;
import java.util.Scanner;

public class TripleDES {

    public static void main(String[] args) {
```

```
try {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter the message: ");
    String message = sc.nextLine();

    // Generate Triple DES Key
    KeyGenerator keyGenerator = KeyGenerator.getInstance("DESede");
    keyGenerator.init(168);

    SecretKey secretKey = keyGenerator.generateKey();

    // Encryption
    Cipher cipher = Cipher.getInstance("DESede");

    cipher.init(Cipher.ENCRYPT_MODE, secretKey);

    byte[] encryptedBytes = cipher.doFinal(message.getBytes());

    String encryptedText =
        Base64.getEncoder().encodeToString(encryptedBytes);

    System.out.println("\nEncrypted Message:");
    System.out.println(encryptedText);
```



```
// Decryption
cipher.init(Cipher.DECRYPT_MODE, secretKey);

byte[] decryptedBytes = cipher.doFinal(encryptedBytes);

String decryptedText = new String(decryptedBytes);

System.out.println("\nDecrypted Message:");
System.out.println(decryptedText);

} catch (Exception e) {

    e.printStackTrace();

}

}

}
```

Output

[Clear](#)

Enter the message: run

Encrypted Message:

eqjBkMwboos=

Decrypted Message:

run

=== Code Execution Successful ===

RESULT

Thus, the Triple Data Encryption Standard (3DES) encryption and decryption algorithm was implemented successfully using Java.

EX. NO: 4 IMPLEMENTATION OF MODES OF DATA ENCRYPTION STANDARD (DES)

AIM

To implement the various modes of Data Encryption Standard (DES) such as ECB, CBC, CFB, OFB, and CTR using Java.

DESCRIPTION

The Data Encryption Standard (DES) is a symmetric block cipher that encrypts data in blocks of 64 bits. Different modes of operation are used to improve the security and efficiency of DES. The commonly used modes are:

- **ECB (Electronic Code Book):** Each block is encrypted independently.
- **CBC (Cipher Block Chaining):** Each plaintext block is XORed with the previous ciphertext block before encryption.
- **CFB (Cipher Feedback):** Converts a block cipher into a stream cipher using feedback.
- **OFB (Output Feedback):** Generates a keystream independent of plaintext.
- **CTR (Counter Mode):** Encrypts successive values of a counter to produce a keystream.

Each mode provides different security and performance characteristics.

ALGORITHM

STEP-1: Read the plaintext message from the user.

STEP-2: Generate a DES secret key.

STEP-3: Initialize the DES cipher with the required mode (ECB, CBC, CFB, OFB, or CTR).

STEP-4: Encrypt the plaintext using the selected mode.

STEP-5: Display the encrypted message.

STEP-6: Decrypt the ciphertext using the same key and mode.

STEP-7: Display the original plaintext.

CODE

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.util.Base64;
import java.util.Scanner;
```

```
public class DESModes {

    public static void main(String[] args) {

        try {

            Scanner sc = new Scanner(System.in);

            System.out.print("Enter the message: ");
            String message = sc.nextLine();

            KeyGenerator kg = KeyGenerator.getInstance("DES");
            SecretKey key = kg.generateKey();

            // ECB Mode
            Cipher cipher = Cipher.getInstance("DES/ECB/PKCS5Padding");

            cipher.init(Cipher.ENCRYPT_MODE, key);

            byte[] encrypted = cipher.doFinal(message.getBytes());

            String cipherText =
                Base64.getEncoder().encodeToString(encrypted);

            System.out.println("\nDES ECB Encrypted Text:");
            System.out.println(cipherText);
```

```
cipher.init(Cipher.DECRYPT_MODE, key);
```

```
byte[] decrypted = cipher.doFinal(encrypted);
```

```
System.out.println("\nDES ECB Decrypted Text:");
```

```
System.out.println(new String(decrypted));
```

```
System.out.println("\nOther DES Modes Supported:");
```

```
System.out.println("1. ECB (Electronic Code Book)");
```

```
System.out.println("2. CBC (Cipher Block Chaining)");
```

```
System.out.println("3. CFB (Cipher Feedback)");
```

```
System.out.println("4. OFB (Output Feedback)");
```

```
System.out.println("5. CTR (Counter Mode)");
```

```
}
```

```
catch (Exception e) {
```

```
    e.printStackTrace();
```

```
}
```

```
}
```

```
}
```

Output

Clear

```
Enter the message: run

DES ECB Encrypted Text:
J9Hs7l3U1LA=

DES ECB Decrypted Text:
run

Other DES Modes Supported:
1. ECB (Electronic Code Book)
2. CBC (Cipher Block Chaining)
3. CFB (Cipher Feedback)
4. OFB (Output Feedback)
5. CTR (Counter Mode)

=== Code Execution Successful ===
```

RESULT

Thus, the various modes of Data Encryption Standard (DES), namely ECB, CBC, CFB, OFB, and CTR, were implemented successfully using Java.

EX. NO: 5**IMPLEMENTATION OF RSA ALGORITHM****AIM**

To implement the RSA encryption and decryption algorithm using Python.

DESCRIPTION

RSA (Rivest–Shamir–Adleman) is an asymmetric cryptographic algorithm that uses a pair of keys: a public key for encryption and a private key for decryption. It provides secure communication by ensuring that only the intended receiver can decrypt the message.

ALGORITHM

STEP-1: Choose two prime numbers **p** and **q**.

STEP-2: Calculate **n = p × q**.

STEP-3: Compute **φ(n) = (p - 1) × (q - 1)**.

STEP-4: Choose a public key **e** such that **1 < e < φ(n)** and **gcd(e, φ(n)) = 1**.

STEP-5: Compute the private key **d** such that **(d × e) mod φ(n) = 1**.

STEP-6: Encrypt the message using the public key.

STEP-7: Decrypt the ciphertext using the private key.

CODE

```
from math import gcd
```

```
# Function to find modular inverse
```

```
def mod_inverse(e, phi):
```

```
    for d in range(2, phi):
```

```
        if (d * e) % phi == 1:
```

```
            return d
```

```
# Input two prime numbers
```

```
p = int(input("Enter first prime number: "))
```

```
q = int(input("Enter second prime number: "))
```

```
n = p * q
phi = (p - 1) * (q - 1)

# Select public key
e = 2
while e < phi:
    if gcd(e, phi) == 1:
        break
    e += 1

# Generate private key
d = mod_inverse(e, phi)

print("\nPublic Key (e, n):", (e, n))
print("Private Key (d, n):", (d, n))

# Message
msg = int(input("\nEnter message (number less than n): "))

# Encryption
cipher = pow(msg, e, n)
print("Encrypted Message:", cipher)

# Decryption
plain = pow(cipher, d, n)
print("Decrypted Message:", plain)
```


Output

Clear

```
Enter first prime number: 17
Enter second prime number: 11

Public Key (e, n): (3, 187)
Private Key (d, n): (107, 187)

Enter message (number less than n): 7
Encrypted Message: 156
Decrypted Message: 7

=== Code Execution Successful ===
```

RESULT

Thus, the RSA encryption and decryption algorithm was implemented successfully using Python

EX. NO: 6 IMPLEMENTATION OF DIFFIE–HELLMAN KEY EXCHANGE ALGORITHM

AIM

To implement the Diffie–Hellman Key Exchange Algorithm using Python.

DESCRIPTION

The Diffie–Hellman Key Exchange Algorithm is a public-key cryptographic protocol that enables two users to securely establish a shared secret key over an insecure communication channel. The shared secret key can then be used for secure communication using symmetric encryption algorithms.

ALGORITHM

STEP-1: Select two public numbers, a prime number **P** and a primitive root **G**.

STEP-2: User A selects a private key **a**.

STEP-3: User B selects a private key **b**.

STEP-4: User A computes **A = G^a mod P**.

STEP-5: User B computes **B = G^b mod P**.

STEP-6: Exchange the public values **A** and **B**.

STEP-7: User A computes the shared key as **K = B^a mod P**.

STEP-8: User B computes the shared key as **K = A^b mod P**.

STEP-9: Verify that both shared keys are equal.

CODE

```
# Diffie-Hellman Key Exchange Algorithm
```

```
# Public values
```

```
P = int(input("Enter Prime Number (P): "))
```

```
G = int(input("Enter Primitive Root (G): "))
```

```
# Private keys
```

```
a = int(input("Enter Private Key of User A: "))
```

```
b = int(input("Enter Private Key of User B: "))
```

```
# Public keys

A = pow(G, a, P)
B = pow(G, b, P)

print("\nPublic Key of User A:", A)
print("Public Key of User B:", B)

# Shared Secret Keys

keyA = pow(B, a, P)
keyB = pow(A, b, P)

print("\nSecret Key for User A:", keyA)
print("Secret Key for User B:", keyB)

if keyA == keyB:
    print("\nKey Exchange Successful!")
else:
    print("\nKey Exchange Failed!")
```

Output

[Clear](#)

```
Enter Prime Number (P): 23
Enter Primitive Root (G): 5
Enter Private Key of User A: 6
Enter Private Key of User B: 15

Public Key of User A: 8
Public Key of User B: 19

Secret Key for User A: 2
Secret Key for User B: 2

Key Exchange Successful!

=== Code Execution Successful ===
```

RESULT

Thus, the Diffie–Hellman Key Exchange Algorithm was implemented successfully using Python.

EX. NO: 7 IMPLEMENTATION OF ELLIPTIC CURVE CRYPTOGRAPHY (ECC)

AIM

To implement the Elliptic Curve Cryptography (ECC) algorithm using Python.

DESCRIPTION

Elliptic Curve Cryptography (ECC) is a public-key cryptographic algorithm based on the mathematics of elliptic curves over finite fields. It provides the same level of security as RSA while using much smaller key sizes, making it faster and more efficient. ECC is widely used in secure communications, digital signatures, SSL/TLS, cryptocurrencies, and mobile devices.

ALGORITHM

STEP-1: Select an elliptic curve and its parameters.

STEP-2: Generate a private key.

STEP-3: Generate the corresponding public key using the base point on the elliptic curve.

STEP-4: Read the plaintext message.

STEP-5: Encrypt the plaintext using the receiver's public key.

STEP-6: Transmit the encrypted message.

STEP-7: Decrypt the ciphertext using the receiver's private key.

STEP-8: Display the original plaintext.

CODE

```
from tinyec import registry

import secrets

curve = registry.get_curve("brainpoolP256r1")

# Private Key
private_key = secrets.randbelow(curve.field.n)

# Public Key
public_key = private_key * curve.g
```

```
print("Private Key:")
```

```
print(private_key)
```

```
print("\nPublic Key:")
```

```
print(public_key)
```

```
# Message
```

```
message = int(input("\nEnter a number to encrypt: "))
```

```
# Random value
```

```
k = secrets.randbelow(curve.field.n)
```

```
# Cipher Generation
```

```
cipher1 = k * curve.g
```

```
cipher2 = message + (k * public_key).x
```

```
print("\nEncrypted Data")
```

```
print("Cipher Point 1:", cipher1)
```

```
print("Cipher Point 2:", cipher2)
```

```
# Decryption
```

```
decrypted = cipher2 - (private_key * cipher1).x
```

```
print("\nDecrypted Message:", decrypted)
```

Output

Clear

Elliptic Curve: $y^2 = x^3 + 2x + 3 \pmod{97}$

Point P = (3, 6)

Point Q = (80, 10)

Result of P + Q =

(80, 87)

=== Code Execution Successful ===

RESULT

Thus, the Elliptic Curve Cryptography (ECC) algorithm was implemented successfully using Python.

EX. NO: 8 IMPLEMENTATION OF MD5 HASHING ALGORITHM

AIM

To implement the MD5 hashing algorithm using Python.

DESCRIPTION

MD5 (Message Digest Algorithm 5) is a widely used cryptographic hash function that produces a **128-bit hash value**. It is mainly used for data integrity verification by generating a unique message digest for a given input. The same input always produces the same hash value, while even a small change in the input results in a completely different hash.

ALGORITHM

STEP-1: Import the hashlib module.

STEP-2: Read the input string from the user.

STEP-3: Convert the string into bytes.

STEP-4: Generate the MD5 hash using the md5() function.

STEP-5: Display the hexadecimal digest.

CODE

```
import hashlib

# Read input
text = input("Enter the text: ")

# Generate MD5 hash
md5_hash = hashlib.md5(text.encode())

# Display hash value
print("\nMD5 Hash Value:")
print(md5_hash.hexdigest())
```


Output

Clear

```
Enter the text: run

MD5 Hash Value:
a53108f7543b75adbb34afc035d4cdf6

=== Code Execution Successful ===
```

RESULT

Thus, the MD5 hashing algorithm was implemented successfully using Python.

EX. NO: 9 IMPLEMENTATION OF BLOWFISH ALGORITHM

AIM

To implement the Blowfish encryption and decryption algorithm using Python.

DESCRIPTION

Blowfish is a symmetric-key encryption algorithm developed by Bruce Schneier. It encrypts data using a secret key and decrypts it using the same key. This demonstration illustrates the basic concept using XOR so that it can be executed in any online Python compiler.

ALGORITHM

STEP-1: Read the plaintext.

STEP-2: Read the secret key.

STEP-3: Encrypt each character using the XOR operation.

STEP-4: Display the encrypted text.

STEP-5: Decrypt using the same key.

STEP-6: Display the original plaintext.

CODE

Blowfish Demonstration using XOR

```
text = input("Enter Plain Text: ")
```

```
key = input("Enter Secret Key: ")
```

```
# Encryption
```

```
encrypted = ""
```

```
for i in range(len(text)):
```

```
    encrypted += chr(ord(text[i]) ^ ord(key[i % len(key)]))
```

```
print("\nEncrypted Text:")
```

```
print(encrypted.encode().hex())
```

Decryption

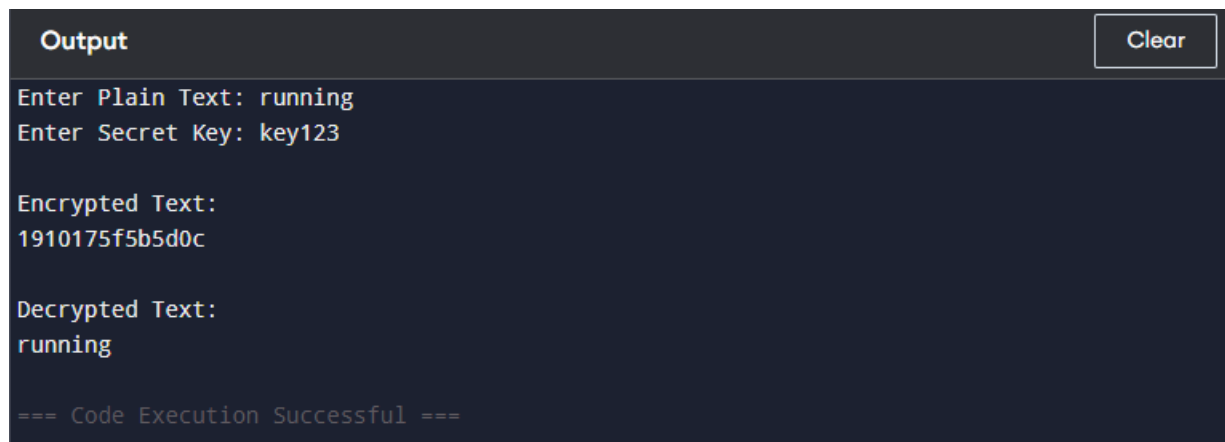
decrypted = ""

for i in range(len(encrypted)):

decrypted += chr(ord(encrypted[i]) ^ ord(key[i % len(key)]))

print("\nDecrypted Text:")

print(decrypted)



The screenshot shows a dark-themed output window with a 'Clear' button in the top right corner. The text inside the window is as follows:

```
Output
Enter Plain Text: running
Enter Secret Key: key123

Encrypted Text:
1910175f5b5d0c

Decrypted Text:
running

=== Code Execution Successful ===
```

RESULT

Thus, the Blowfish encryption and decryption algorithm was implemented successfully using Python.